

CENG 467 — Technical Checkpoint Report

Natural Language Understanding and Generation

İzmir Institute of Technology, Spring 2026

Project Title: Schema-Aware Instruction Tuning for Text-to-SQL Semantic Parsing

Group Members: [Ad Soyad 1] [Ad Soyad 2]

Submission Date: May 14, 2026

Project Title, Group Members, and Problem Definition

This project addresses the task of **Text-to-SQL semantic parsing**, which involves automatically translating natural language questions into executable SQL queries over relational databases.

The task is important because it enables non-expert users to query structured databases without knowledge of SQL syntax, a capability with broad practical applications in business intelligence, healthcare, and data analysis.

We work on the **Spider** [?] benchmark, a large-scale cross-domain dataset containing [8,659] training and [1,034] development question–SQL pairs across 200 databases. The input to the system is a natural language question paired with a database schema; the output is a syntactically valid and semantically correct SQL query.

The task is technically challenging for several reasons. First, the model must correctly identify which tables and columns are relevant to the question (*schema linking*). Second, it must generalize to database schemas unseen during training, requiring compositional reasoning rather than pattern memorization. Third, SQL supports complex constructs such as nested subqueries, aggregations, and multi-table joins that are difficult to generate reliably.

Dataset / Benchmark Status

Property	Value
Dataset name	Spider
Source	https://yale-lily.github.io/spider
Training samples	8,659 question–SQL pairs
Development samples	1,034 question–SQL pairs
Number of databases	200 (138 domains)
Input format	Natural language question + database schema
Output format	SQL query string
Train/Dev/Test split	7,000 / 1,034 / 2,147 (test labels not public)

Data Source. The Spider dataset is publicly available and was downloaded from the official Yale Semantic Parsing repository. The dataset includes `train_spider.json`, `dev.json`, `tables.json` (schema definitions), and a `database/` directory containing SQLite files for execution-based evaluation.

Input/Output Format. Each data instance contains a natural language question, a database identifier, and a gold SQL query. Schema information (table names, column names, primary keys, and foreign keys) is stored separately in `tables.json` and must be joined with each question at preprocessing time.

Preprocessing Steps Completed.

- Dataset downloaded and directory structure verified.
- Schema loading script implemented: `tables.json` is parsed and mapped to each question by `db_id`.
- Three schema serialization formats (plain text, CREATE TABLE syntax, compact notation) have been defined and prototyped.
- Tokenization pipeline is under development using the HuggingFace `AutoTokenizer` for the selected model.

Known Data Challenges. The Spider training set exhibits class imbalance across SQL complexity levels; simple single-table queries are overrepresented relative to nested queries involving multiple JOINS and subqueries. Additionally, the test set labels are not publicly released, so all evaluation is performed on the development split.

Literature Review Progress

Our literature review focuses on three areas: neural Text-to-SQL models, schema encoding strategies, and instruction tuning for semantic parsing.

Spider Benchmark. Yu et al. (2018) introduced Spider as the first large-scale cross-domain Text-to-SQL benchmark, defining the task formulation and the execution accuracy metric used in subsequent work.

Sequence-to-Sequence Approaches. [Scholak et al. (2021) tarafından önerilen PICARD yaklaşımını 1-2 cümle açıklayın. Örn: "PICARD augments a T5 model with an incremental SQL parser that constrains beam search to only produce syntactically valid SQL tokens, significantly improving execution accuracy on Spider."]

Pre-trained Language Models. [Seçtiğiniz modelin Text-to-SQL'e nasıl uygulandığını 2-3 cümle açıklayın.]

Instruction Tuning. [DAIL-SQL veya başka bir instruction-tuning tabanlı Text-to-SQL çalışmasını 1-2 cümle özetleyin.]

Schema Linking. [RAT-SQL'in şema-soru ilişkisini nasıl modellediğini 1-2 cümle açıklayın.]

Summary. Based on the reviewed literature, the dominant direction for competitive Text-to-SQL performance combines pre-trained sequence-to-sequence models with explicit schema encoding and instruction tuning. Our project follows this direction by fine-tuning [MODEL ADI] with structured schema serialization on Spider, treating schema encoding format as an experimental variable.

Baseline Models or Prompting Strategies

We plan to implement two baseline approaches that represent distinct paradigms for Text-to-SQL generation.

Baseline 1 — Zero-shot Prompting. We will evaluate a large instruction-following language model (e.g., `Mistral-7B-Instruct`) in a zero-shot setting. The model receives the serialized database schema and the natural language question as a prompt and is asked to generate the SQL query without any task-specific fine-tuning. This baseline establishes the upper bound achievable through prompting alone and serves as a reference for the value added by fine-tuning.

Suitability: Zero-shot prompting is a standard and widely reported baseline in recent Text-to-SQL literature, making it a meaningful point of comparison.

Baseline 2 — Fine-tuned Seq2Seq Model. The second baseline is a pre-trained sequence-to-sequence model fine-tuned on Spider without our proposed schema encoding strategy. We will use a publicly available checkpoint (e.g., `tscholak/cxmefzzi` on HuggingFace) to represent the performance of standard fine-tuning, against which our schema-aware instruction tuning will be compared.

Suitability: This baseline isolates the contribution of our schema serialization strategy by holding the model class constant and varying only the input representation.

Baseline	Approach	Model	Status
Baseline 1	Zero-shot prompting	Mistral-7B-Instruct	In progress
Baseline 2	Fine-tuned Seq2Seq	T5 / PICARD checkpoint	In progress
Proposed	Instruction tuning + schema encoding	Flan-T5-base	Planned

Initial Experimental Results

At this stage of the project, we have [completed initial setup / begun running preliminary experiments] and report the following results on a subset of the Spider development set.

Model / Strategy	EX (%)	EM (%)	Status
Baseline 1: Zero-shot prompting	–	–	In progress
Baseline 2: Fine-tuned Seq2Seq	–	–	In progress
Proposed: Instruction tuning	–	–	Planned

Baseline experiments are currently being set up. Preliminary runs on a 100-sample subset of the Spider development set are planned for the coming days. We expect to report initial execution accuracy figures for both baselines before the final submission.

Planned Improvements and Technical Direction

Our primary improvement over the baselines is **schema-aware instruction tuning**: fine-tuning a sequence-to-sequence language model (Flan-T5-base or Flan-T5-large) on the Spider training set while providing explicit, structured schema representations as part of the input.

The planned technical steps are as follows:

1. **Instruction template design:** Finalize the prompt format that includes task instruction, serialized schema, and natural language question.
2. **Schema serialization comparison:** Implement and evaluate three serialization formats (plain text listing, CREATE TABLE syntax, compact notation) to identify the most effective representation.
3. **Fine-tuning:** Train Flan-T5-base on Spider using the HuggingFace `Trainer` API with AdamW optimizer, cosine learning rate scheduling, and early stopping on development execution accuracy.
4. **Inference and evaluation:** Run the fine-tuned model on the Spider development set and compute execution accuracy by executing predicted SQL against the SQLite databases.
5. **Comparison:** Systematically compare the proposed method against both baselines and across schema format variants.

If computational resources allow, we will also experiment with LoRA-based parameter-efficient fine-tuning to reduce GPU memory requirements.

Ablation or Prompt Sensitivity Plan

Our ablation study will systematically isolate the contribution of the schema serialization format, which we consider the central design choice of our system. Specifically, we will train and evaluate identical model configurations (same architecture, hyperparameters, and training data) while varying only the schema representation:

- **Format A** — plain text listing of tables and columns;
- **Format B** — CREATE TABLE SQL syntax (our primary format);
- **Format C** — compact notation with primary/foreign key annotations.

Additionally, if time permits, we plan to compare:

- zero-shot vs. few-shot prompting for Baseline 1;
- model size effect: Flan-T5-base (250M) vs. Flan-T5-large (780M);
- with vs. without foreign key information in the schema representation.

Error Analysis and Generation Quality Analysis

For the final report, we plan to manually inspect a sample of prediction errors from the Spider development set and categorize them by failure type. Based on the Text-to-SQL literature, we anticipate the following error categories:

- **Schema linking errors:** incorrect table or column selection;
- **JOIN errors:** missing or incorrect JOIN conditions and ON clauses;
- **Aggregation errors:** incorrect use of COUNT, SUM, AVG, GROUP BY, or HAVING;
- **Nested query failures:** errors in subquery structure;
- **WHERE clause errors:** incorrect filter conditions or string values.

For each category, we will provide (1) the count and percentage of failures, (2) a representative example showing the gold SQL and the predicted SQL, and (3) a brief explanation of why the model failed. We also note that evaluating generation quality in Text-to-SQL differs from open-ended generation: a syntactically fluent query may still be semantically incorrect, which motivates our use of execution-based evaluation as the primary metric.

Ethical Considerations and Bias

Deploying Text-to-SQL systems in real-world applications raises several responsible AI concerns that we acknowledge in our project.

Hallucination risk. A model may generate syntactically valid SQL that references non-existent columns or tables, silently returning incorrect results. Users without SQL expertise cannot easily detect such errors, making hallucination particularly risky in this context.

Domain bias. The Spider benchmark is English-only and skews toward standard business domains. Models trained exclusively on Spider may perform poorly on domain-specific databases (e.g., medical, legal) or on queries formulated in other languages, raising fairness concerns for non-English-speaking users.

Misuse scenarios. A Text-to-SQL system integrated into a database-facing application could potentially be misused to extract sensitive information if query generation is not sandboxed or restricted by appropriate access controls.

GitHub and Reproducibility Status

Repository: [https://github.com/\[KULLANICI_ADJ\]/\[REPO_ADJ\]](https://github.com/[KULLANICI_ADJ]/[REPO_ADJ])

Current Repository Structure:

```
text-to-sql-ceng467 /
|-- README.md           # Project overview and setup instructions
|-- requirements.txt     # Python dependencies
|-- data/
|   '-- README.md       # Spider download instructions
|-- notebooks/
|   '-- (experiments planned)
'-- src/
    '-- (model scripts planned)
```

Reproducibility Status:

- **README:** Initialized with project title, team members, and dataset download instructions. *[In progress / Complete]*
- **requirements.txt:** Lists core dependencies (torch, transformers, datasets, sqlparse, evaluate). *[In progress / Complete]*
- **Dataset preparation:** Instructions for downloading Spider from the official repository are documented in `data/README.md`.
- **Commit history:** Repository initialized; regular commits will be maintained as development progresses.

Current Challenges and Next Steps

Current Challenges.

- **Computational resources:** Fine-tuning even a 250M-parameter model on the full Spider training set requires significant GPU memory. We are currently evaluating whether Google Colab Pro or a local GPU is sufficient, or whether parameter-efficient methods (LoRA) are needed.
- **Baseline selection:** We are finalizing the choice of the second baseline, weighing the tradeoff between using a publicly available checkpoint (faster) versus training a simple model from scratch (more educational).
- **Execution-based evaluation setup:** Correctly executing generated SQL against 200 different SQLite databases requires careful path management and error handling, which is currently being implemented.

Next Steps (before final submission).

1. Finalize baseline selection and obtain initial execution accuracy results on the Spider development set. *(Week 12)*
2. Implement instruction tuning pipeline for Flan-T5-base with all three schema serialization formats. *(Week 12–13)*
3. Run ablation experiments comparing schema formats. *(Week 13)*
4. Perform manual error analysis on 20+ failure cases. *(Week 13–14)*
5. Write the LNCS-format final report (6–8 pages). *(Week 14–15)*
6. Prepare Gradio demo and finalize GitHub repository. *(Week 15)*

References

- [1] Yu, T., Zhang, R., Yang, K., et al.: Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In: Proceedings of EMNLP, pp. 3911–3921 (2018).

- [2] Scholak, T., Schucher, N., Bahdanau, D.: PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In: Proceedings of EMNLP, pp. 9895–9901 (2021).
- [3] Chung, H.W., et al.: Scaling Instruction-Finetuned Language Models. arXiv preprint arXiv:2210.11416 (2022).
- [4] Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In: Proceedings of ACL, pp. 7567–7578 (2020).